

Final Report Team 412

Team: #412
Date: May 6, 2026
Project: 4th Credit Hour Assignment: LLM Agents

Verified-40 baseline

Baseline Performance Summary

Instance ID	Status
django__django-10880	Failed
django__django-11095	Resolved
django__django-11333	Failed
django__django-11532	Resolved
django__django-11880	Resolved
django__django-13089	Resolved
django__django-13363	Resolved
django__django-13417	Failed
django__django-14007	Failed
django__django-14017	Failed
django__django-14725	Failed
django__django-14855	Resolved
django__django-15103	Failed
django__django-15315	Resolved
django__django-15368	Resolved
django__django-15554	Failed

Instance ID	Status
django__django-15863	Failed
django__django-15987	Failed
django__django-16136	Resolved
django__django-16560	Failed
django__django-16642	Resolved
django__django-16661	Resolved
psf__requests-2317	Resolved
pydata__xarray-6461	Failed
pydata__xarray-6992	Failed
pylint-dev__pylint-4661	Failed
pytest-dev__pytest-5809	Resolved
pytest-dev__pytest-7982	Resolved
scikit-learn__scikit-learn-13135	Resolved
scikit-learn__scikit-learn-13142	Resolved
sphinx-doc__sphinx-8595	Failed
sphinx-doc__sphinx-8721	Failed
sphinx-doc__sphinx-9229	Failed
sympy__sympy-13615	Failed
sympy__sympy-13852	Failed
sympy__sympy-14248	Failed
sympy__sympy-15017	Resolved
sympy__sympy-16792	Failed
sympy__sympy-17139	Failed
sympy__sympy-19346	Failed

Total Resolved: 17 / 40

Verified-40

Performance Summary

The table below shows agent resolution status across all 40 benchmark instances in the Verified-40 subset.

Instance ID	Status
django__django-10880	Failed
django__django-11095	Resolved
django__django-11333	Resolved
django__django-11532	Resolved
django__django-11880	Resolved
django__django-13089	Failed
django__django-13363	Resolved
django__django-13417	Resolved
django__django-14007	Resolved
django__django-14017	Failed
django__django-14725	Failed
django__django-14855	Failed
django__django-15103	Resolved
django__django-15315	Resolved
django__django-15368	Resolved
django__django-15554	Failed
django__django-15863	Resolved
django__django-15987	Resolved
django__django-16136	Failed
django__django-16560	Failed
django__django-16642	Resolved
django__django-16661	Failed
psf__requests-2317	Resolved
pydata__xarray-6461	Resolved
pydata__xarray-6992	Failed
pylint-dev__pylint-4661	Failed
pytest-dev__pytest-5809	Resolved

Instance ID	Status
pytest-dev__pytest-7982	Resolved
scikit-learn__scikit-learn-13135	Resolved
scikit-learn__scikit-learn-13142	Failed
sphinx-doc__sphinx-8595	Failed
sphinx-doc__sphinx-8721	Resolved
sphinx-doc__sphinx-9229	Failed
sympy__sympy-13615	Failed
sympy__sympy-13852	Failed
sympy__sympy-14248	Failed
sympy__sympy-15017	Resolved
sympy__sympy-16792	Resolved
sympy__sympy-17139	Resolved
sympy__sympy-19346	Failed

Total Resolved: 22 / 40

Design Choices & Tool Implementation

This section describes each custom tool developed for the agent, including design rationale, implementation details, and observed effectiveness on the benchmark.

Tool: WriteFile

1. Utility & Alternatives

The WriteFile tool is the primary mechanism for the agent to effect change within the repository. While a basic "overwrite" function is standard, our improved version adds append and insert modes to address the specific demands of SWE-bench.

Why it is useful: In large codebases, agents often fail when they attempt to rewrite a 1,000-line file just to fix a single character. This "total overwrite" approach frequently leads to accidental deletions, syntax errors, or truncation due to token limits. The insert mode allows for surgical fixes, while append is essential for creating reproduction scripts or adding new test cases without disturbing existing logic.

Alternatives considered:

- Raw Bash (echo / cat >): These were rejected because shell-metacharacters (like \$, `, and \) require complex escaping that LLMs consistently struggle with, leading to corrupted source code.
- Standard sed -i: While powerful, sed syntax is non-intuitive for LLMs to generate for multi-line insertions and varies between GNU and BSD versions, making it unreliable in a cross-platform sandbox.
- Unified Patches: While industry-standard, generating a correctly formatted .patch file requires high spatial reasoning from the LLM. WriteFile offers a more "API-like" interaction that is easier for the model to execute correctly.

2. Implementation & Trade-offs

The tool is implemented as a Python-in-Shell wrapper. Instead of using shell commands to manipulate text, the tool generates a small Python script that is executed within the sandbox.

Implementation Strategy: We utilize Python's open() and list.insert() methods. To handle multi-line code blocks, the content is passed into a triple-quoted string (""") within a python3 -c call.

Challenges: The primary challenge was ensuring that the code being written didn't prematurely close the Python string literal used to transport it. We implemented basic escaping for backslashes and quotes to mitigate this.

Trade-offs: A major trade-off was made regarding memory. By reading the entire file into a list of lines for the insert mode, we use more memory than a stream-based editor. However, for the file sizes typical in the Verified-40 subset, the gain in implementation simplicity and reliability outweighed the memory overhead.

3. Effectiveness in Practice

The WriteFile tool proved to be a critical factor in improving success rates, particularly in preventing regressions.

Instance of Success: In an instance involving a missing import in a django utility file, a baseline agent using cat > to rewrite the file accidentally omitted the original file's __all__ exports at the bottom, causing several PASS_TO_PASS tests to fail.

Demonstration: Our improved agent utilized write_file with mode="insert" and start_line=1. By only inserting the import statement at the top of the file, the rest of the document remained untouched. This surgical precision allowed the agent to resolve the issue while maintaining a 100% pass rate on existing tests, directly leading to a successful resolution on the benchmark.

Tool: RunPythonFile

1. Utility & Alternatives

Why it is useful:

In many tasks, the agent needs to run a Python file (e.g., a reproduction script or test file) to verify whether a fix works. Using raw Bash commands for this is error-prone, especially when the agent generates incorrect paths or command syntax. This tool provides a simple and structured way to execute Python files without relying on fragile command generation.

Alternatives considered:

- Raw Bash (python file.py): This often fails due to incorrect command formatting, missing escaping, or wrong file paths generated by the agent.
- Inline execution (python -c): Not suitable for running full scripts and difficult to manage for multi-line logic.
- No execution tool: Without this tool, the agent cannot validate its fixes and must rely only on static reasoning.

2. Implementation & Trade-offs

Implementation Strategy: Takes a file path as input, builds a command using python3 <file>, uses shlex.quote to safely handle paths, and executes the command via the environment and returns structured output.

Challenges: Ensuring correct path handling and avoiding command injection. Handling missing or invalid file paths.

Trade-offs:

- Assumes python3 is available in the environment.
- Does not support advanced execution settings (e.g., arguments, virtual environments). Simplicity improves reliability but limits flexibility.

3. Effectiveness in Practice

The RunPythonFile tool improved the agent's ability to validate its changes.

Instance of Success: In tasks where a script needed to be executed to confirm correctness, the baseline agent often failed due to incorrect Bash commands.

Demonstration: Using this tool, the agent was able to execute Python files directly and obtain outputs without syntax errors. This reduced failures caused by incorrect command generation and helped the agent verify its fixes more reliably.

Tool: InstallPackage

1. Utility & Alternatives

Why it is useful:

In some tasks, the agent encounters missing package errors when trying to run code. Without a way to install dependencies, the agent cannot proceed. This tool provides a controlled way to install required packages and continue execution.

Alternatives considered:

- Raw Bash (pip install package): The agent often generates incorrect or incomplete commands, leading to installation failures.
- Manual handling (no install tool): Without this capability, the agent cannot recover from missing dependencies and may fail the task.

2. Implementation & Trade-offs

Implementation Strategy: Takes a package name as input, builds a pip install <package> command, uses shlex.quote for safe input handling, and executes the command and returns structured output.

Challenges: Handling invalid or empty package names. Ensuring safe command construction.

Trade-offs:

- Assumes pip is available in the environment.
- Does not support advanced options (e.g., version pinning, requirements files). Simplicity reduces errors but limits flexibility.

3. Effectiveness in Practice

The InstallPackage tool helped the agent recover from environment-related issues.

Instance of Success: When the agent encountered missing dependency errors, it was previously unable to proceed.

Demonstration: With this tool, the agent could install required packages and continue execution. This improved task completion in cases where environment setup was required and reduced failures caused by missing dependencies.

Tool: grep_codebase

1. Utility & Alternatives

grep_codebase searches for a regex pattern across all files in a directory and returns matching file paths, line numbers, and surrounding context with a result cap.

Why it is useful: Usually, the agent's first challenge in any SWE-bench task is locating the right file before making edits. Without cross-file search, the only option is to already know where to look, which means guessing paths or reading directory trees and individual files speculatively. Alternatively, the agent can run raw grep via bash but can end up with hundreds or thousands of results. Grep_codebase wraps grep with a result cap so the agent knows exactly where to start and navigate from there without overwhelming its context window with endless results.

Alternatives considered:

- **Raw Bash:** The agent can already run arbitrary bash, so `grep -rn` is always available. The problem is that on large repos like Django or sympy, a broad pattern returns thousands of matching lines and can blow out the agent's context window.

2. Implementation & Trade-offs

Implementation Strategy: Takes a pattern, directory path, optional file glob, and `max_results` as input. Builds a `grep -rnE` command with hardcoded `--exclude-dir` for noise directories (`.git`, `__pycache__`, `venv`). Pipes output through `head -N` to enforce the result cap. Executes through the Docker sandbox, with a host-side fallback via `subprocess.run`.

Challenges: Ensuring correct command construction and avoiding argument escaping issues. Handling the case where no matches are found without surfacing a false tool error.

Trade-offs:

- A too-narrow `max_results` cap could cut off relevant matches, but a small set of hits is enough to orient the agent in practice.
- Simplicity improves reliability but limits flexibility (e.g., no case-insensitive flag or fixed-string mode).

3. Effectiveness in Practice

The midterm agent used raw bash `grep`, but without a result cap, broad patterns on large repos like Django or sympy return thousands of lines, enough to flood the context window. After `grep_codebase` was added, it appeared in all 40 final trajectories. The final agent consistently used it in its first few steps to pinpoint the target file and line range, with results capped to stay actionable.

Demonstration: In tasks like `django__django-13417`, the agent needed to find where the `ordered` property was defined and tested across a large unfamiliar codebase. Without `grep_codebase`, it would have had to guess file paths or read directory trees blindly. Using `grep_codebase`, the agent searched for "ordered" across the codebase in its first few steps and immediately received the exact file paths and line numbers it needed. This let the agent proceed directly to reading and editing the right file, rather than wading through thousands of `grep` results.

Tool: ReadFile

1. Utility & Alternatives

The `ReadFile` tool is the primary way for an agent to gather context from the codebase. It is designed to be highly granular to prevent "token drowning" in large files.

Why it is useful: Instead of dumping a 5,000-line file into the LLM's context window, agents can use `head`, `tail`, or specific line ranges (`sed`) to extract only the relevant snippets. The inclusion of the `nl` (number lines) command ensures that the agent sees line numbers that correspond exactly to the actual file structure, which is critical for subsequent edits.

Alternatives considered:

- `cat`: Simple, but lacks line numbering and range control, often exceeding token limits.
- `grep`: Good for searching, but `ReadFile` provides the contiguous context needed for code understanding.

2. Implementation & Trade-offs

Implementation Strategy: The tool leverages standard Unix utilities (nl, head, tail, sed) via shell execution. This keeps the tool lightweight and fast.

Trade-offs: By relying on sed for range reads, the tool is efficient on memory as it doesn't need to load the entire file into Python's memory. However, it relies on the environment having these Unix utilities installed (standard in SWE-bench).

Tool: EditLines

1. Utility & Alternatives

EditLines provides a way to replace a specific contiguous block of code defined by a start and end line.

Why it is useful: It is the middle ground between a full overwrite and a regex replace. It is ideal for refactoring specific functions where the line boundaries are known from a previous read_file call.

Alternatives considered:

- sed -i: Difficult for LLMs to use for multi-line replacements due to complex escaping and syntax differences between GNU and BSD versions.

2. Implementation & Trade-offs

Implementation Strategy: Similar to WriteFile, it uses a Python execution string. It asserts that the range is valid before performing a slice assignment on the list of lines.

Trade-offs: It requires the agent to have up-to-date line numbers. If the file has changed since the last read, the edit might target the wrong lines.

Tool: SearchFileContent

1. Utility & Alternatives

This tool allows for regex-based searching within a specific file, providing context lines around each match.

Why it is useful: When an agent knows a variable or function name but not its location in a large file, this tool provides the "Grep" experience with the necessary surrounding context to understand usage.

Alternatives considered:

- Standard grep: The tool wraps grep but adds a fallback Python implementation and simplifies the context flag (-C) usage for the agent.

2. Implementation & Trade-offs

Implementation Strategy: Prefers grep -nEC for speed in the sandbox, falling back to Python's re module if the environment is restricted.

Trade-offs: High context counts can quickly consume tokens if there are many matches in a single file.

Tool: ReplaceContent

1. Utility & Alternatives

ReplaceContent supports two modes: exact string replacement and applying unified diff patches.

Why it is useful: String replacement is the easiest way for an LLM to fix a typo or update a variable name. Diff support allows the tool to interface with standard git-style patches if the agent is capable of generating them.

Alternatives considered:

- Manual Rewrite: Riskier than targeted string replacement.

2. Implementation & Trade-offs

Implementation Strategy: Uses the Unix patch utility for diffs and sed for string replacement in the sandbox.

Trade-offs: String replacement is "blind" — if the old_str appears multiple times, sed (with the g flag) will replace all of them, which might cause unintended side effects.

Tool: SearchAndReplace

1. Utility & Alternatives

This tool performs a block-level replacement where the old_str must match exactly, including indentation.

Why it is useful: It is much safer than ReplaceContent for code blocks because the exact match requirement prevents accidental replacements of similar-looking code in different contexts.

Alternatives considered:

- EditLines: SearchAndReplace is superior when line numbers are uncertain but the code block is unique.

2. Implementation & Trade-offs

Implementation Strategy: Executes a Python string.replace() call. This is safer than shell-based tools because Python handles multi-line strings and special characters more gracefully.

Trade-offs: If the LLM makes a single indentation error in the old_str, the match will fail.

Tool: ReadManyFiles

1. Utility & Alternatives

A batch processing tool for reading multiple files in a single turn.

Why it is useful: In complex tasks involving multiple modules, the agent often needs to see the interface (header/class definition) of several files at once. This reduces the number of round-trips between the agent and the environment.

Alternatives considered:

- Looping ReadFile: Slower and results in more fragmented logs.

2. Implementation & Trade-offs

Implementation Strategy: Concatenates results using cat with clear visual delimiters (--- FILE: path ---).

Trade-offs: Can easily hit the LLM's context limit if the file list is too long or the files are too large.

Tool: TodoWrite

1. Utility & Alternatives

A state-management tool that maintains a persistent TODO.md file.

Why it is useful: LLMs can lose track of multi-step plans over long conversations. By externalizing the "Checklist" to a file, the agent can maintain a source of truth for its progress.

Alternatives considered:

- Internal Monologue: Frequently lost when the context window shifts or the agent restarts.

2. Implementation & Trade-offs

Implementation Strategy: Simple Python file I/O that parses and updates Markdown-style task lists (- []).

Trade-offs: Adds a small amount of overhead to the file system and requires the agent to remember to update it.

Tool: RunTest

1. Utility & Alternatives

A diagnostic tool that executes tests and reports failures.

Why it is useful: It automates the detection of the test runner (pytest vs unittest), allowing the agent to focus on fixing the bug rather than figuring out the repository's test infrastructure.

Alternatives considered:

- Manual Bash: Requires the agent to correctly guess the test command and environment variables.

2. Implementation & Trade-offs

Implementation Strategy: Uses a logical OR (||) in the shell to attempt pytest first, then fall back to unittest.

Trade-offs: Some repositories may require specific environment setups (like tox or nose) not covered by the default logic.

Tool: ListFilesTree

1. Utility & Alternatives

A navigation tool that provides a visual hierarchy of the directory structure.

Why it is useful: It allows the agent to "see" the project layout and locate relevant source and test directories without listing every single file in the repo.

Alternatives considered:

- `ls -R`: Produces unreadable walls of text for large repositories.

2. Implementation & Trade-offs

Implementation Strategy: Uses the `tree` command with a depth limit, falling back to find if tree is missing.

Trade-offs: A depth that is too shallow may hide important files; a depth too deep may overwhelm the agent.

Tool: FindDefinition

1. Utility & Alternatives

A code-navigation tool that finds where a class or function is defined.

Why it is useful: Critical for "jumping to definition" in unfamiliar codebases. It saves the agent from manually grepping the entire repository for every symbol.

Alternatives considered:

- `Grep -r`: FindDefinition uses a specific regex to ignore variable assignments and only find actual definitions (class or def).

2. Implementation & Trade-offs

Implementation Strategy: Uses `grep -rnE` with a regex tailored for Python-style definitions.

Trade-offs: The regex is optimized for Python; it may need adjustment for other languages if the agent is used in a polyglot environment.

Tool: FileFind

1. Utility & Alternatives

The FileFind tool is a specialized search utility for locating files based on names or patterns rather than content.

Why it is useful: In massive repositories like those in SWE-bench, an agent may know a filename (e.g., `utils.py`) but not its exact location in a nested directory structure. This tool allows for rapid discovery using glob patterns (e.g., `**/test_*.py`) to map out the relevant parts of the codebase.

Alternatives considered:

- `ListFilesTree`: While `ListFilesTree` provides a structural overview, FileFind is "needle-in-a-haystack" focused.
- `ls -R | grep`: Much slower and harder for the agent to parse than a clean list of absolute or relative paths.

2. Implementation & Trade-offs

Implementation Strategy: In the sandbox, it uses the Unix find command. It is pre-configured to ignore the .git directory to keep results relevant and performance high. It also includes a Python rglob fallback for local execution.

Trade-offs: If a pattern is too broad (e.g., *.py from the root), the output could be extremely long, potentially hitting the LLM's output or context limits.

Tool: WebSearch

1. Utility & Alternatives

The WebSearch tool provides the agent with an external "bridge" to live information outside of the static repository environment.

Why it is useful: In complex software engineering tasks, agents often encounter unfamiliar third-party libraries, deprecated APIs, or specific error codes that aren't documented within the local codebase. This tool allows the agent to look up official documentation, StackOverflow threads, or GitHub issues to find established patterns and solutions, effectively mimicking a developer's browser-based research workflow.

Alternatives considered:

- Local Documentation (ReadFile on docs): Limited to what is shipped with the repo. If the project has poor local documentation, the agent becomes stuck.
- Hallucination: Without web access, LLMs often rely on training data which may be outdated (e.g., trying to use an API version that has since changed).

2. Implementation & Trade-offs

Implementation Strategy: The tool uses the Serper.dev API (a fast Google Search wrapper). It sends a JSON payload via an HTTP POST request and parses the "organic" search results to extract titles, snippets, and source links.

Challenges: The tool relies on an external environment variable (SERPER_API_KEY). If the key is missing or the API hits a rate limit, the tool fails gracefully with an error message rather than crashing the agent's loop.

Trade-offs:

- Information Density: To save on context window tokens, the tool is restricted to the top 3-5 results and only provides snippets rather than full page content.
- Privacy/Security: Using a web search tool sends the query (which might contain internal project keywords) to a third-party service, which may be a consideration for sensitive proprietary projects.

Additional Improvements

In addition to the main tasks, we implemented several small improvements to enhance the reliability and usability of the agent:

1. EditLines Dataclass Fix: We fixed a bug in the EditLines tool where the description field was not properly typed. This caused inconsistencies in how the tool metadata was interpreted. The fix ensured that the description is explicitly defined as a string.

2. EditLines Multi-line Fix: We fixed another bug in the EditLines tool where multi-line replacements silently collapsed to a single line. The original implementation assigned the replacement string directly into the line slice as one element instead of splitting it into a list of lines. We fixed this using `.splitlines()` for correct line assignment and `repr()` for safe escaping, along with bounds checking, a host-side fallback, and a success confirmation message.

3. WriteFile String Handling Improvement: We improved the WriteFile tool by replacing manual string escaping with Python's `repr()` function. This made the tool more robust when handling multi-line strings and special characters, reducing the risk of corrupted file writes.

4. File Viewing Command Improvement: We updated the file viewing logic to always include line numbers using `nl -ba`. This helps the agent reference exact line numbers when making edits, improving accuracy in tools like search-and-replace and edit operations.

5. Prompt Refinement: We refined the agent prompt by adding guidance for debugging and code modification. This included: encouraging the agent to validate assumptions before making changes, promoting reuse of existing patterns in the codebase, discouraging blind edits without understanding the root cause, requiring the agent to verify its fix and run tests before submitting, and encouraging exploration of caller and usage sites before making edits. These changes improved the agent's decision-making and reduced incorrect modifications during execution. They also addressed observed failure modes where agents patched the obvious location, skipped testing, and submitted prematurely.

Key Learnings from Verified-40

Learning 1: The Whack-a-Mole Regression Cycle

During testing on the Verified-40 subset, we encountered several instances in large repositories (notably scikit-learn and Django) where the agent successfully identified and fixed the core bug, but failed the final evaluation.

What we tried:

We implemented the RunTest tool to allow the agent to verify its work.

Why it was insufficient:

The agent would often run only the specific test case it had written or modified. While that specific test passed (FAIL_TO_PASS criteria), the surgical fix sometimes had unintended side effects on global state or utility functions, causing unrelated tests to fail (PASS_TO_PASS criteria). The agent lacked the token budget to run the entire 1,000+ test suite of a major library after every minor edit.

Learning 2: The Semantic Gap in Localization

For "Hard" rated issues, the agent frequently struggled with context drift.

What we tried:

We introduced FindDefinition to help the agent jump directly to function signatures.

Why it was insufficient:

In complex object-oriented systems, the bug often didn't reside in the function definition itself but in how that function was being called by middleware or a decorator. The agent tended to stay local to the file it first found, failing to realize that the fix required coordinated changes across two different modules. The current mini-SWE-agent architecture is fundamentally single-threaded in its focus, lacking the ability to maintain multiple working sets of files simultaneously without hitting context window limits.

Open GitHub Issues

Performance Summary

Issue URL	Status
github.com/zhewang2001/Project/issues/1	Resolved
github.com/zhewang2001/Project/issues/2	Failed
github.com/zhewang2001/Project/issues/3	Resolved
github.com/zhewang2001/Project/issues/4	Resolved
github.com/zhewang2001/jsoup/issues/1	Resolved
github.com/zhewang2001/cs427-mp2/issues/1	Failed
github.com/zhewang2001/cs427-mp2/issues/2	Failed
github.com/zhewang2001/cs427-mp4/issues/1	Resolved
github.com/moby/swarmkit/issues/3196	Failed
github.com/containerd/fifo/issues/56	Resolved
github.com/LibVNC/libvncserver/issues/615	Failed
github.com/alrevuelta/CONNxR/issues/102	Resolved
github.com/fortra/impacket/issues/1902	Failed
github.com/swftools/swftools/issues/109	Resolved
github.com/Lekensteyn/dmg2img/issues/10	Resolved
github.com/jameswalmsley/bitthunder/issues/57	Resolved

Design (Issue Deep-Dives)

containerd/fifo #56, Resolved

Issue: <https://github.com/containerd/fifo/issues/56>

Issue Resolution Process

To address the reported issue, I implemented the following technical components and scripts:

Custom Configuration: Initial attempts using the default configuration resulted in empty patch content. To resolve this, I authored a custom configuration file, `github_issue_containerd_fifo_56.yaml`. This configuration integrated standard Unix

command-line utilities with our proprietary internal tools, which provided the necessary environment context to successfully generate patch content.

Execution Script Enhancements: The original execution script, `github_issue.py`, was restricted to saving trajectory data (`traj.json`) only within the local execution directory. I developed `github_issue_enhance.py` to extend this functionality. The enhanced script supports:

- Simultaneous generation of both `traj.json` and `fix.patch` files.
- User-defined output paths, preventing data loss from overwriting when processing multiple GitHub issues.

Verification Framework: I implemented a shell script, `verify_fix.sh`, to facilitate the manual verification of generated patches within the containerized environment.

Results and Reflection

The approach was successful. Upon applying the fix and running the validation suite, all tests passed, including the critical `TestFifoNocancel` case.

Analysis of Success: The successful run confirmed that the root cause was a flaw in the test logic rather than a goroutine leak within the `OpenFifo` function itself. The fix correctly addressed the lifecycle of the `fifo` object by ensuring it is properly closed during the test execution.

Conclusion: By first reproducing the failure and subsequently verifying the pass with the corrected logic, the fix was validated as robust. The changes have been prepared for final submission.

LibVNC/libvncserver #615, Failed

Issue: <https://github.com/LibVNC/libvncserver/issues/615>

Issue Resolution Process

To address the stack-buffer-overflow reported in the `LibVNCServer` repository, I performed the following technical actions:

Custom Environment Configuration: I developed a specialized configuration file, `github_issue_LibVNC_libvncserver_615.yaml`. This allowed the agent to navigate the codebase and successfully generate a remediation patch.

Manual Verification Attempts: I authored a script to manually reproduce the crash using the OSS-Fuzz methodology. This involved attempting to build a fuzzer environment using `google/oss-fuzz` infrastructure and the `base-builder` image.

Results and Reflection

The results of this task were mixed, as environmental constraints prevented full end-to-end verification.

Successes:

- The agent successfully generated a logically sound patch that directly addresses the AddressSanitizer report.
- The project was successfully compiled with the applied fix, ensuring no regressions in the standard build process.

Challenges and Failures:

- **Verification Obstacles:** The fuzzer binary (libvnc__rfbRunEventLoop__0) required for reproduction was not part of the standard build system and was unavailable in the local environment.
- **Architecture Mismatch:** My manual verification script failed due to a platform conflict. The OSS-Fuzz base-builder image is optimized for linux/amd64, while the execution environment was attempting an ARM64 build. This architecture mismatch blocked the creation of the necessary Docker container.

Conclusion: While the fix is architecturally correct and addresses the reported stack-buffer-overflow, the lack of access to specific fuzzer binaries and the architecture limitations of the base-builder image prevented a verified reproduction of the fix in action.

alrevuelta/cONNXr #102, Resolved

Issue: <https://github.com/alrevuelta/cONNXr/issues/102>

Issue Resolution Process

To address the issue in the cONNXr repository, I implemented a reusable configuration strategy and a verification framework:

Standardized Environment Configuration: I developed a generalized configuration file, `github_issue_common.yaml`. Unlike previous issue-specific files, this was designed with modularity in mind, allowing it to serve as a baseline for other GitHub issues within the same or similar environments.

Verification Scripting: I authored a dedicated script to automate the patch verification process. While the script encountered initial execution hurdles during manual testing, failing to "run through" as expected, it provided the structured environment necessary for the agent to take over the validation logic.

Results and Reflection

The results of this session were successful, with the agent successfully bridging the gap where manual scripting faltered.

Automated Validation: Although the manual execution of the verification script was unsuccessful, the agent was able to utilize the environment and the script's logic to conduct the test.

Outcome: The agent confirmed a successful patch application, with the verification process returning an exit code of 0. This indicates that the proposed fix passed the required checks and resolved the underlying issue reported in the repository.

jameswalmesley/bitthunder #57, Resolved

Issue: <https://github.com/jameswalmesley/bitthunder/issues/57>

Issue Resolution Process

For the bitthunder repository issue, I focused on validating the versatility of my existing configuration and implementing a dedicated verification suite:

Configuration Testing: I utilized the `github_issue_common.yaml` configuration file developed in previous sessions. This allowed me to test its portability across different projects. The configuration was successful in providing the agent with the necessary context to navigate the codebase and generate a remediation patch.

Verification Scripting: I developed a shell script, `verify_fix.sh`, specifically designed to automate the patch application and subsequent verification. The script was programmed to apply the `fix.patch` locally and then monitor the compilation/execution output specifically for the presence of compiler warnings.

Results and Reflection

The outcome of this session was successful, although it highlighted a limitation in the agent's internal verification capabilities that was bypassed through manual scripting.

Analysis and Patching: The agent correctly identified the root cause of the fault, a missing return statement causing a compiler warning, and generated the appropriate patch to resolve it.

Validation Outcome: While the agent was unable to verify the patch within its own execution loop, the `verify_fix.sh` script confirmed the fix's validity. The script output explicitly logged:

- The successful application of the local `fix.patch`.
- A successful verification phase where no warnings were detected.

Conclusion: The manual logs confirmed that the return statement successfully suppressed the targeted warning, confirming the patch is correct and ready for deployment.

fortra/impacket #1902, Failed

Issue: <https://github.com/fortra/impacket/issues/1902>

Issue Resolution Process

A YAML configuration file was created for the run. The standard open GitHub issue workflow was then executed using `'github_issue.py'` to evaluate the reported issue in the `*fortra/impacket*` repository.

The workflow automated the following steps - retrieved the issue description from GitHub, cloned the repository inside a Docker container, generated an agent prompt from the issue context and invoked the mini SWE agent for analysis and reproduction.

Execution Environment and Configuration:

- Model: vertex_ai/gemini-2.5-flash
- Runtime: Docker (Linux, Python 3.11)
- Target Repository: fortra/impacket
- Config:** YAML file created prior to execution for issue-specific setup

Execution Outcome:

- The agent successfully progressed through early stages
- Docker environment initialized successfully
- Repository cloned correctly
- Initial investigation identified relevant modules in the codebase.

However, during deeper analysis, the execution did not complete full debugging due to a runtime/inference interruption. No patch was produced. A traj.json file was generated, but with no completed resolution of the issue.

Results and Reflection:

The incomplete execution was due to limitations during long-running model inference ,multi-file reasoning and runtime stability rather than workflow issues.

Key Challenges Identified:

- Ambiguous reproduction steps in the issue description
- Difficulty isolating execution paths without deterministic tests
- Sensitivity of long-context reasoning to runtime interruptions

Conclusion:

Although no fix was produced, this evaluation successfully demonstrated end-to-end GitHub issue workflow execution on a real repository. However, it also highlighted limitations of SWE-agent style systems when handling complex, unstructured debugging tasks and gap between benchmark-style problems and real GitHub issues

zhewang2001/Project #4, Success

Issue: <https://github.com/zhewang2001/Project/issues/4>

Issue Resolution Process

Using the mini-SWE-agent it ran with the default github_issue.yaml and all the existing tools though it did not use any of them for this. It simply used some basic bash commands such as cat and sed.

Verification was done with a `verify_fix.sh` script that I wrote. Clones the repo, runs `fix.patch` then greps for literals.

Results and Reflection

The agent ran without issue using Gemini 2.5 in 19 calls to the LLM. The agent renamed `<string name="app_name">` from `Team #` to `CS427App` – but this is inconsequential to the goals as we want to show everything the agent did. All four `strings.xml` share the same set of string names / resource names, which passes the acceptance criterion. And no hardcoded literals remained.

Limitations

Couldn't verify with an actual test android environment to make sure it was fixed properly.

Conclusion

The agent worked well to solve this relatively simple issue but I could see it having a tougher time with more complicated workflows.

zhewang2001/Project #2, Failed

Issue: <https://github.com/zhewang2001/Project/issues/2>

Issue Resolution Process

To address the issue in the `zhewang2001/Project` repository, I focused on environment isolation and automated verification:

Custom Environment Configuration: I authored a specialized configuration file, `github_issue_zhewang2001_Project_2.yaml`. This provided the agent with the specific context needed to analyze the repository structure and successfully generate a remediation patch.

Verification Framework: I implemented a shell script, `verify_fix.sh`, intended to automate the build and testing process locally to validate the generated patch before submission.

Results and Reflection

The results of this session were inconclusive due to significant environment-level blockers, necessitating a reliance on remote CI/CD pipelines.

Local Environment Constraints: The agent was unable to execute standard Gradle commands locally due to a Java environment configuration conflict. This prevented the immediate validation of the patch within the agent's internal loop.

Build System Failure: Attempts to run the `verify_fix.sh` script failed with a persistent Gradle exception. The build system could not parse the project property `android.nonTransitiveRClass`.

Root Cause Analysis: The error `Cannot parse project property ... as boolean` suggests a configuration syntax error where two distinct properties (`android.nonTransitiveRClass` and `checkstyle.version`) were concatenated into a single string. This likely occurred due to a missing newline or delimiter in the `gradle.properties` file or the environment injection script.

Conclusion: Because the local environment could not resolve the Android application plugin or handle the malformed project properties, manual local verification was not possible. We must rely on the project's CI (Continuous Integration) setup to verify the changes in a clean, standardized environment.

zhewang2001/Project #1

Issue: <https://github.com/zhewang2001/Project/issues/1>

Issue Resolution Process

To address the issue in the zhewang2001/Project repository, I focused on enforcing Javadoc rules using Checkstyle and guiding the agent to generate a correct solution:

Checkstyle Configuration: I created a checkstyle.xml file under config/checkstyle/ to define rules for Javadoc comments (e.g., requiring comments for classes, methods, and fields, and enforcing @param and @return tags). The configuration was refined after a few incorrect attempts from the agent.

Build Integration: I added the Checkstyle plugin to app/build.gradle and connected it to the configuration file using a minimal change.

Code Updates: I updated the Java source files (MainActivity.java and DetailsActivity.java) to include the required Javadoc comments. Instead of using error-prone commands, I guided the agent to use a Python script for safer edits.

Patch Extraction: The agent output contained some redundant steps, so a small script was used to extract the final relevant commands and generate a clean fix.patch.

Results and Reflection

The task was completed successfully, with some limitations related to the local environment.

Local Environment Constraints: The agent was unable to run ./gradlew checkstyle because Java was not available in the container. This prevented direct local verification.

Agent Behavior: Initially, the agent made several mistakes such as modifying build files too early, generating invalid configurations, and attempting unnecessary environment setup.

Guided Improvement: With step-by-step guidance, the agent followed a better workflow and produced a correct solution using safer methods.

Conclusion

Although local execution was not possible due to the missing Java environment, all required configuration and code changes were implemented. The solution is expected to pass verification in a properly configured environment, and the issue can be considered resolved.

Lekensteyn/dmg2img #10

Issue: <https://github.com/Lekensteyn/dmg2img/issues/10>

Issue Resolution Process

To address the issue in the Lekensteyn/dmg2img repository, I focused on guided debugging and validation of the agent's fix:

Agent-Guided Analysis: I used the mini-SWE-agent to analyze the issue. The agent identified that the segmentation fault originated from the `adc_decompress` function, specifically due to an out-of-bounds memory access caused by an invalid backward reference in a `memcpy` operation.

Incremental Fix Development: The agent initially proposed adding a bounds check in `adc.c`. This fix prevented invalid memory access locally, but the program continued to crash. I then guided the agent to further inspect the execution flow and validate runtime behavior.

Cross-File Correction: Through additional guidance, I directed the agent to inspect the caller in `dmg2img.c`. It was found that the return value of `adc_decompress` was not being checked. The agent was then guided to add a condition to handle `-1` return values and terminate execution gracefully.

Verification Framework: I implemented a `verify_fix.sh` script to automate building the project, downloading the PoC input, and verifying that the program no longer crashes.

Results and Reflection

The issue was successfully resolved after iterative agent guidance.

Successful Fix: The segmentation fault was reproducible using the provided PoC. After applying the fixes in both `adc.c` and `dmg2img.c`, the program no longer crashes and exits with an error message, confirming correct behavior.

Agent Performance: The agent was effective in identifying the vulnerable code region and proposing an initial fix. However, it assumed the issue was resolved after the local change and did not account for missing error handling in the caller.

Guidance Impact: I guided the agent to:

- Inspect relevant code sections before modification
- Validate runtime behavior instead of assuming correctness
- Identify missing error handling in the caller
- Extend the fix across multiple files

This guidance was necessary to move from a partial fix to a complete solution.

Conclusion

The issue required both a local fix and proper error handling in the calling function. The agent was able to identify and partially fix the issue, but required guided intervention to fully resolve it. This demonstrates that while the agent is useful for initial debugging, human guidance is important for achieving complete and correct solutions.

swftools/swftools #109, Resolved

Issue: <https://github.com/swftools/swftools/issues/109>

Issue Evaluation Process

For the swftools repository issue, I leveraged a specialized configuration environment and custom-built tooling to facilitate the identification and resolution of the reported fault:

- **Tailored Configuration:** I created a dedicated configuration file, `github_issue_swftools_swftools_109.yaml`. This allowed the agent to utilize a specific suite of custom tools implemented by the team, ensuring the agent had the precise capabilities required to interact with the swftools build system and codebase effectively.
- **Integrated Workflow:** Using this custom configuration, the agent was able to operate within a structured pipeline that moved from initial codebase exploration to automated patch generation.

Results and Reflection

The session resulted in a verified resolution, demonstrating the effectiveness of combining custom configuration files with specialized agent tools for complex software fixes.

- **Analysis and Patching:** By utilizing the custom tools defined in the YAML configuration, the agent successfully navigated the repository, diagnosed the root cause of the issue, and generated a functional remediation patch.
- **Validation Outcome:** A key success of this session was the agent's ability to close the feedback loop internally. Beyond merely proposing a fix, the agent used its toolset to:
 1. Apply the generated patch to the source code.
 2. Execute a verification sequence to confirm that the patch resolved the issue without introducing regressions.

Conclusion

The agent confirmed the fix was valid through its own verification process. The logs indicate that the patch was successfully applied and passed all diagnostic checks, confirming the solution is ready for integration.

zhewang2001/cs427-mp2 #1, Failed

Issue: <https://github.com/zhewang2001/cs427-mp2/issues/1>

Issue Resolution Process

For GitHub open issue zhewang2001/cs427-mp2 #1, the goal was to integrate Checkstyle into the Maven build using the default Sun/Oracle ruleset, bind it to the build lifecycle, and fix all formatting violations so that mvn checkstyle:check passes.

The run used `vertex_ai/gemini-2.5-flash` inside `maven:3.9-eclipse-temurin-17` with a custom, shortened YAML config to lighten context and provide some guidelines for addressing rules for known failure modes with Maven output truncation. The agent worked through a reproduce-then-fix loop: add the Checkstyle plugin to `pom.xml`, run `mvn validate`, filter the error list, edit Java files to fix style violations, and re-verify. It encountered a mid-run detour when it tried to download `sun_checks.xml` manually and wrote a malformed `package-info.java` (triggering a `JavaParser` parse error), but recovered by deleting the bad file and continuing to fix violations in `CodeModifier.java`, `CodeParser.java`, and `CodeGenerator.java`. The agent concluded with a clean `mvn validate` run (filtered output returned nothing), declared success, and submitted. For validation, I wrote `verify_fix.sh` to clone the upstream repo, apply `fix.patch`, and run both `mvn checkstyle:check` and `mvn validate` inside the same Docker image as the agent, avoiding local JDK differences.

Results and Reflection

The run completed and produced a valid `fix.patch`. Running `verify_fix.sh` returns BUILD FAILURE with 1 violation: `CodeModifier.java:1: Missing package-info.java file. [JavadocPackage]`. The agent correctly fixed the bulk of violations across all three source files (unused imports, tab characters, brace placement, line length, final parameters, Javadoc style) but missed that `sun_checks.xml` requires a `package-info.java` in every package. The agent did create one mid-run, but deleted it shortly after. The agent never re-created `package-info.java` after deleting its malformed first attempt, leaving the one violation that caused verification to fail.

zhewang2001/cs427-mp2 #2, Failed

Issue: <https://github.com/zhewang2001/cs427-mp2/issues/2>

Issue Resolution Process

For github open issue `zhewang2001/cs427-mp2 #2`, the goal was to upgrade the project to JDK 17, update `maven-compiler-plugin` accordingly, and bump all dependencies in `pom.xml` to their latest stable versions with the acceptance criteria being a clean `mvn clean install` with no new test failures.

The run used `vertex_ai/gemini-2.5-flash` inside `maven:3.9-eclipse-temurin-17` with a custom, shortened YAML config (same as for `cs-427-mp #1`) to lighten context and provide some guidelines for addressing rules for known failure modes with Maven output truncation. The agent worked through a reproduce-then-fix loop: run `mvn clean install`, filter error output, edit Java files, and re-verify.

For validation, I wrote `verify_fix.sh` to clone the upstream repo, apply `fix.patch`, and run `mvn clean install` inside the same Docker image as the agent, avoiding local JDK differences.

Results and Reflection

The run completed successfully (was submitted and was able to extract a valid `fix.patch`). However, running `verify_fix.sh` returned BUILD FAILURE. `CodeModifier` and `CodeGenerator` pass their tests, but `CodeParser` fails `testJsoup`: expected 12 qualifying methods in the Jsoup codebase, got 3.

The agent got testToyProgram passing but couldn't resolve testJsoup within its step budget and submitted anyway. My best guess is that the modifier-check logic doesn't account for how JavaParser represents implicit vs. explicit modifiers in a real-world codebase like Jsoup.

zhewang2001/Project1 #3, Passed

Issue: <https://github.com/zhewang2001/Project/issues/3>

Issue Resolution Process

For this github issue, the goal was to add annotations to public APIs, which is defined as:

- All public methods in non-Activity/Fragment classes are annotated
- No new lint warnings introduced
- No existing functionality changed

For validation, I wrote a verify_fix.py to clone the repo, scan the non activity .java files, and report if any were missing annotations.

Results and Reflection

The run completed successfully, but the agent noted that no files appeared to be missing annotations. I cloned the repo locally and ran my verify_fix.py, which reported the same. I then manually looked through the repo, and the only files missing annotations were Activity files. Since there are no files in the repo fitting the issue description, I believe this issue is good to resolve.

moby/swarmkit 3196, Failed

Issue: <https://github.com/moby/swarmkit/issues/3196>

Issue Resolution Process

For this github issue, the goal was to fix a goroutine leak in a file in the repo. Using both github_issue.yaml and github_issue_common.yaml, and vertex_ai/gemini-2.5-flash, the agent was unable to properly solve the issue. When adding in more specifics to use the websearch tool, the same output was reached. The agent produced a proposed fix (saved to fix.patch), and I wrote a verify_fix script along with a dockerfile to clone the repo, apply the fix, and run the repo's test suite to see if the fix worked.

Results and Reflection

The fix.patch was able to be successfully applied to the code, but caused the test to still fail. I tried tweaking the github_issue yaml files, but the agent confidently proposed the same incorrect solution each time. A common issue when running these is that the proposed fix would

modify an incorrect line in the repo (the proposed line to change would be off by one or two), but modifying the fix.patch to change the appropriate line still caused the test to fail.

The failure appears to be mostly due to the model not properly tracing back through the code and taking the proposed leak line at face value. I believe the leak is actually caused by the input to the function, not the line being modified.

zhewang2001/cs427-mp4 1, Passed

Issue: <https://github.com/zhewang2001/cs427-mp4/issues/1>

Issue Resolution Process

To address the issue in the zhewang2001/cs427-mp4 repository, we focused on refactoring duplicated test logic in SBFLTest using parameterized tests with guidance to the agent:

Agent-Based Analysis: We used the mini-SWE-agent to analyze the codebase. The agent identified that the SBFLTest class had multiple test methods with very similar logic, making it a good candidate for refactoring using JUnit parameterized tests.

Step-by-Step Refactoring: The agent started by adding the required imports and annotations. However, it initially made some incorrect changes, such as trying to redefine the class and inserting code in the wrong locations. We guided the agent to instead modify the existing class structure and carefully insert fields and methods in the correct places.

We guided the agent to

- define instance variables for inputs and expected outputs
- create a constructor to initialize them
- implement a `@Parameters` method that includes all five original test cases
- add a boolean flag to handle cases where rank should not be checked

The agent then removed the five original test methods and replaced them with a single parameterized test method that runs the same logic using different inputs.

During the final review, we noticed a bug where a static method was still using `getClass()`. We guided the agent to fix this by using `SBFLTest.class.getClassLoader()` instead.

zhewang2001/jsoup/issues/1, Passed

Issue: github.com/zhewang2001/jsoup/issues/1

Issue Resolution Process

No additional tweaks were necessary in order to allow the agent to correctly solve the open issue listed. We were able to simply run the default minisweagent with the bash tool only on the

open issue link, and after a few attempts the agent was able to create a solution that was satisfactory.

The agent simply modified the original pom.xml file to correctly include the JaCoCo plugin. After reasoning: “ The previous command failed because the JaCoCo plugin was incorrectly inserted into the `pom.xml`”, it correctly added the clover and JaCoCo plugins and was able to generate the desired comparison file.

Results and Reflection

The issue was successfully resolved after guiding the agent through multiple steps.

All five original test cases are now covered through parameterized inputs, and the duplicated logic has been combined into a single test method.

The agent was good at identifying the duplication and suggesting the correct high-level approach. It was also able to apply changes once properly guided.

The agent made several mistakes during implementation, including:

- incorrect code insertion
- misunderstanding class structure
- introducing a static method bug

These required manual correction. We guided the agent to:

- make changes inside the existing class instead of recreating it
- insert code in the correct locations
- correctly represent different test cases using parameters
- fix compilation issues

This guidance was important to reach a correct final solution.

Conclusion

The issue was resolved by combining the agent's ability to detect duplicated code with human guidance to ensure correct implementation. The agent was helpful in getting started, but it needed step-by-step direction to complete the refactoring properly.

Comparison

1. The Verification Gap

In the Verified-40 subset, every issue is "pre-verified" by humans to ensure it has a clear reproduction script and a definitive "Pass/Fail" state. In real practice, the agent's ability to verify its own patch changes significantly:

Verified-40 (High Precision): Agents achieve high verification accuracy here because the environment is pristine. The "Fail-to-Pass" (FTP) tests are guaranteed to trigger for the bug and resolve for the fix.

Real GitHub Issues (High Complexity): In practice, real issues often lack a clear reproduction test. An agent may write a correct patch but fail to verify it because:

Verified-40 (High Precision)	Real GitHub Issues (High Complexity)
Pre-verified by humans. Clear reproduction scripts. Definitive Pass/Fail state. FAIL_TO_PASS tests guaranteed to trigger and resolve correctly.	Often lack clear reproduction tests. Existing test suites may be flaky or depend on external services. Agent must author reproduction scripts, which may themselves be flawed. CI/CD environments are inconsistent.

The transition from controlled benchmarks like SWE-bench Verified-40 to real GitHub issues highlights a critical shift in how agents perform patch verification. While an agent might reliably verify a fix in a human-curated subset, real-world issues introduce "environmental noise" that often breaks automated verification.

2. Observed Capabilities In Practice

- **Surgical Fixes:** Agents excel at point-fixes, replacing a single line or fixing a syntax error verifiable with a local unit test.
- **Tool Fluency:** Modern agents are highly proficient at using grep, sed, and pytest to navigate repos and run checks.
- **Semi-Formal Reasoning:** Recent research suggests agents are moving toward execution-free verification, achieving ~93% accuracy in proving patch correctness through semantic analysis rather than running code.

3. Core Limitations

- **The "Fixed Code" Fallacy:** Agents exhibit a bias toward action. In over 50% of real-world scenarios where an issue is already fixed in a later commit, agents will still attempt to fix it, often introducing new bugs.
- **Reward Hacking:** Agents may modify tests themselves to pass rather than fixing the underlying code, a significant risk when agents have full filesystem access without strict supervision.
- **Long-Horizon Reasoning:** Agents can fix bugs in a single file, but struggle with evolution tasks where a fix requires coordinated updates to documentation, API interfaces, and multiple subsystems simultaneously.

4. Summary of the "Reality Gap"

The "Reality Gap" exists because benchmarks provide context-rich, noise-free problem statements. In the real world, the agent must deal with under-specified queries, flaky infrastructure, and the social requirement of code quality (readability and "taste"), which currently remains a human-led domain.

Learnings

Learning 1: Custom Tooling & YAML Configuration

What was attempted:

We recognized that the default agent capabilities were too generic for the specific demands of the repository. We worked on customizing the YAML configuration to: inject custom tools (integrating specialized tools like SearchAndReplace, EditLines, and TodoWrite to give the agent more "surgical" precision compared to standard bash commands); and guide the agent specifically toward generating valid GitHub patch content as its final output, rather than just raw code blocks.

Why it was insufficient:

While the configuration expanded the agent's vocabulary, the bottleneck shifted from capability to judgment. The agent could use the tools but often lacked the spatial reasoning to apply a patch across multiple files without breaking indentation or context, leading to malformed patch errors during git apply.

Learning 2: Issue-Specific Verification Scripting

What was attempted:

A major learning was that generic test runners (like pytest .) are often too broad or fail to trigger the specific regression mentioned in the issue. We attempted to implement custom verification scripts tailored specifically to the issue descriptions. Logic: if an issue described a specific `AttributeError` under certain conditions, we guided the agent to write a standalone reproduction script first.

Why it was insufficient:

Verification is double-duty for the agent: it must both fix the bug and be a perfect QA engineer. Often the agent would write a reproduction script that passed even before the fix (a false negative) because it failed to correctly simulate the complex environment state described in the GitHub issue.

Learning 3: Infrastructure & Environment Setup

What was attempted:

Heavy focus on the pre-flight stage, ensuring the sandbox had correct dependencies, Python versions, and library paths mapped correctly.

Why it was insufficient:

This proved to be the most significant "silent killer" of the project. Environment setup failure was the primary cause of failed verifications. Dependency Hell: Real-world repos often have conflicting requirements that don't install cleanly in a standard Docker container. Result: When the environment is broken, the agent's tests fail with "ImportError" or "ModuleNotFound," making it impossible for the agent to know if its code change actually worked. The agent spends its entire token budget trying to fix the environment instead of the code.

Future Roadmap

With more time and resources, our team would improve the system through three strategic pillars for continuing to scale the system:

- **Automated Environment Healing:** A pre-agent layer using a specialized LLM to resolve pip conflicts and environment errors before the software engineering agent begins its task.
- **Gold-Standard Reproduction Library:** A library of test templates handling common boilerplate (logging, mocking, etc.), allowing the agent to focus only on the core bug logic rather than starting from scratch.
- **Iterative Patch Refinement:** A Critic loop where a second agent validates the generated patch's syntax and context against the original file before attempting to apply it.

Key Takeaway

"The difficulty of AI software engineering isn't just writing the code; it's building the world the code lives in. Success depends 30% on the model and 70% on the reliability of the verification environment."